

Capstone Project Report: Open Source Software Audit using Git NAME- Manish Rathore Reg No. – 24BAI10931

Author: [Manish Rathore] **Date:** [31 march 2026] **Institutional Affiliation:** [vit Bhopal university]

Document Purpose: A comprehensive analysis of Git's origins, licensing, technical integration, and its role within the Free and Open Source Software (FOSS) ecosystem.

2. Introduction

In the contemporary landscape of computing, **open source software** has evolved into a fundamental pillar of global infrastructure. A significant portion of modern technological frameworks—including **operating systems**, **web servers**, and **development tools**—is constructed upon software that is collaboratively developed and freely distributed. This model is essential for promoting innovation while ensuring transparency, flexibility, and community-driven improvement.

This report provides a formal audit of Git, a **distributed version control system (VCS)** that has fundamentally revolutionized software development practices. The scope of this analysis encompasses the philosophical foundations and technical characteristics of Git, its origin, its **licensing** framework, and its strategic position within the broader **open-source ecosystem**. Furthermore, the project demonstrates technical proficiency through the practical implementation of Git within **Linux** environments.

3. The Genesis and Philosophy of Git

3.1 Historical Context and Origin

The development of Git was initiated in 2005 by Linus Torvalds, the creator of the **Linux kernel**. The impetus for its creation was a critical necessity following the loss of access to **BitKeeper**, a proprietary version control system previously utilized by the **Linux** community. At that juncture, existing alternatives were insufficient for the requirements of large-scale, distributed software development. Torvalds engineered Git to address these gaps, prioritizing **performance**, **data integrity**, and a **distributed architecture**.

3.2 Core Philosophy

Git is predicated on a **distributed model**, a significant architectural departure from centralized systems. In this framework, every developer maintains a complete local copy of the **repository**. Architecturally, this shift was designed to eliminate the serial development bottlenecks and frequent merge conflicts common in centralized environments. This model enables efficient offline work and facilitate **parallel collaboration**.

The introduction of Git successfully addressed specific technical limitations inherent in previous systems:

- **Performance Latency:** Enhancing the speed of versioning operations.
- **Restricted Access:** Removing the bottlenecks associated with centralized authorization.
- **Scalability Issues:** Ensuring the system could handle the demands of massive, multi-contributor codebases.

By releasing Git as **open-source software**, Torvalds established a foundation that allowed a global community of developers to contribute to its ongoing evolution. **4.**

Ethical and Philosophical Framework of Open Source

The open-source model is rooted in the concept of "standing on the shoulders of giants," a philosophy that emphasizes leveraging existing knowledge to drive further innovation. This approach does not limit originality; rather, it fosters an environment where creativity and efficiency coexist through shared knowledge.

There is a strategic balance between collective progress and commercial interests within this ecosystem. Major organizations, such as **Google** and **Meta**, extensively utilize **open-source** technologies like **Linux** and Git. While these corporations generate profit from these tools, they maintain a **mutually beneficial relationship** by contributing back to the community, which serves as a vital governance model for the industry.

Ethically, while **open source** is a powerful driver of advancement, proprietary access remains a necessity in specific sectors. Domains such as security-sensitive systems or proprietary business applications may require controlled access where the **opensource** model is not practical.

5. Licensing Analysis: GNU General Public License (GPL) v2

Git is distributed under the **GNU General Public License (GPL) version 2**. This license is specifically designed to guarantee user freedom while ensuring that the software remains open and transparent.

5.1 The Four Essential Freedoms

The **GPL version 2** is predicated on four essential freedoms granted to the user:

1. The freedom to run the program for any purpose.
2. The freedom to study how the program works.
3. The freedom to modify the software.
4. The freedom to redistribute copies, including modified versions.

5.2 Copyleft vs. Permissive Licenses

Git enforces the concept of **copyleft**, a legal mechanism requiring that any modified version of the software, if distributed, must also be released under the same license. This ensures long-term transparency and prevents organizations from privatizing community-driven innovations by distributing modified versions of Git as proprietary software.

In contrast, permissive licenses such as the **MIT License** allow for the incorporation of **open-source** code into proprietary systems without the obligation to share modifications. While the **MIT License** provides certain flexibilities, it may limit **community benefit**. From a governance perspective, the **GPL version 2** model is superior for promoting collaborative development and preserving openness.

5.3 Conceptual Distinction

A critical distinction exists between "free as in freedom" and "free as in cost." Git exemplifies the former; it is not merely a zero-cost tool but a system that empowers users with absolute control over the software, representing a "freedom" that is central to the **FOSS** movement.

6. Technical Implementation: The Linux Footprint

6.1 System Integration

Git integrates seamlessly into **Linux** environments and is typically installed using standard **package managers** such as **apt**. The installation process is straightforward, requiring minimal system-level configuration to become operational.

6.2 File System and Configuration

The technical footprint of Git on a **Linux** system follows established conventions:

- **Executable Path:** The **binary** is typically located in the `/usr/bin` directory, ensuring system-wide accessibility.
- **Global Configuration:** System-wide settings are stored in `/etc/git`.
- **Local Configuration:** User-specific settings are stored in **the user's home directory** as `.gitconfig`.

6.3 Operational Model

Unlike many server-based applications, Git does not maintain centralized log files; instead, it tracks changes within individual **repositories**. From an architectural standpoint, Git is

lightweight as it does not require a dedicated **background system service** or **daemon**. Instead, it operates under the permissions of the current user. While this simplifies usage, it necessitates diligent permission management in multuser environments.

7. Git’s Role in the FOSS Ecosystem

7.1 Collaborative Infrastructure

Git serves as the primary engine for major collaborative platforms such as **GitHub** and **GitLab**. These platforms build upon Git’s core functionality to provide **version control hosting**, collaboration tools, and project management features.

7.2 Technology Stack Support

While Git is not a direct component of the **LAMP** stack (**Linux, Apache, MySQL, PHP**), it plays a critical supporting role. It is the primary tool used to manage the source code for applications developed and deployed within the **LAMP** framework.

7.3 Community Dynamics

The Git community is global and highly active. It relies on decentralized communication channels, including **mailing lists**, forums, and online **repositories**. This structure ensures continuous development and that **software updates** and security patches are delivered via **Linux package management** systems.

8. Comparative Analysis: Open Source vs. Proprietary Software

The following table compares Git against proprietary version control tools across five key dimensions:

Dimension	Git (Open Source)	Proprietary Tools
Cost	Free	Paid
Security	Transparnt and auditable	Closed source
Support	Community-driven	Vendor-based
Flexibility	High (modifiable)	Limited
Control	User-controlled	Vendor-controlled

Synthesis of Comparison Git’s transparency, efficiency, and reliability make it a superior choice for real-world deployment. While proprietary tools may provide **dedicated customer service** and structured support systems, Git’s open nature encourages continuous improvement and adaptability that proprietary models cannot replicate. The choice of Git, backed by the **GPL version 2** framework, represents a **strategic governance choice** that prioritizes long-term flexibility and user control over vendor-dependent support.

9. Conclusion and Synthesis

This audit demonstrates that open collaboration can produce highly efficient, reliable, and globally adopted technological solutions. The study of Git reveals how a **distributed architecture** and a robust licensing framework can revolutionize an entire industry by fostering a balance between individual contribution and collective progress.

Through the practical execution of this audit, proficiency in **Linux** environments and **shell scripting** has been reinforced—skills that are vital for modern software development. Ultimately, the principles of **open-source** software remain central to shaping the future of technological advancement, ensuring that the global developer community can continue to innovate without the constraints of proprietary silos.